SMM Reseller Panel: Project Documentation

1. Overview

A reseller SMM (social media marketing) panel in the style of indiansmartpanel.com. The panel does not fulfil orders itself. It resells services from upstream provider panels (e.g. JustAnotherPanel) through their HTTP API, adds a markup, and sells them to end customers who pay from a prepaid wallet.

```
Customer --> Our Panel --> Provider API (JAP, others) --> delivery
             |  wallet, markup, orders, tickets
             +-- Our own /api/v2 for the customers' resellers
```

Business flow

1. Customer registers and tops up the wallet (UPI / Razorpay / crypto).
2. Customer picks a service, enters link and quantity.
3. Panel validates, deducts the charge (provider rate x markup), and stores the order.
4. A background job forwards the order to the provider (`add`).
5. A scheduled job polls the provider (`status`) and updates the order. Cancelled or partial orders are refunded to the wallet.

> Compliance note: these services are against the terms of Instagram, YouTube, and others. Payment gateways (Razorpay etc.) may reject or freeze SMM merchants. Make the client aware before launch.

---

2. Recommended Tech Stack

Layer	Choice	Notes
Backend	Laravel 11 (PHP 8.2+)	Auth, queues, scheduler built in
Database	MySQL 8 / MariaDB	ACID needed for wallet and ledger
Cache / Queue	Redis	Order dispatch and status polling jobs
Admin panel	Filament	Services, users, orders, providers
User frontend	Blade + Livewire (or Next.js)	Dashboard, new order, add funds
Payments	Razorpay / Cashfree / UPI, optional crypto	Wallet top-ups
Web server	Nginx + PHP-FPM on a VPS	Shared hosting is not enough for queues and cron
Security	Cloudflare, SSL, admin 2FA, rate limiting, Fail2ban	

Local development: XAMPP (PHP and MySQL) works. Run `php artisan queue:work` and `php artisan schedule:work` alongside.

Alternative: install a ready-made PHP script (SmartPanel / Perfect Panel) and only configure and brand it. This is cheaper and faster, but less flexible.

---

3. Core Modules

1. **Auth and users**: register, login, email verification, API key per user, roles (customer, reseller, admin).
2. **Wallet and ledger**: every balance change is a ledger row (top-up, order charge, refund, adjustment). Never edit the balance without a ledger entry.
3. **Providers**: stores API URL and key per provider, balance check, enable or disable.

4. **Service catalogue**: categories, services mapped to a provider service ID, markup rule (percent or fixed), min, max, refill and cancel flags, sync from provider.
5. **Order engine**: create, dispatch to provider, poll status, refund on cancel or partial.
6. **Refill and cancel**: customer requests are forwarded to the provider.
7. **Payments**: gateway integration with webhooks and idempotent crediting.
8. **Tickets / support**.
9. **Own public API** (`/api/v2`): same format as the provider API so customers' resellers can plug in.
10. **Admin**: dashboards, provider balance alerts, logs, settings.

---

4. Suggested Database Schema

```
users              id, name, email, password, api_key, balance, role, status
providers          id, name, api_url, api_key, currency, balance, active
categories         id, name, sort
services           id, category_id, provider_id, provider_service_id, name, type,
                   provider_rate, markup_type, markup_value, rate, min, max,
                   refill, cancel, active
orders             id, user_id, service_id, link, quantity, runs, interval,
                   charge, provider_cost, provider_order_id, start_count, remains,
                   status, error, created_at, updated_at
refills            id, order_id, provider_refill_id, status
ledger             id, user_id, type, amount, balance_after, ref_type, ref_id, note
payments           id, user_id, gateway, gateway_ref, amount, status, raw_payload
tickets, ticket_messages
settings           key, value
```

Rules:

- Store `provider_rate` and the customer `rate` separately so profit is always visible.
- `charge = rate * quantity / 1000`.
- Wrap "deduct balance + create order" in a single DB transaction.

---

5. Order Lifecycle

Our status	Meaning
Pending	Saved, not yet sent to provider
Processing	Provider accepted it (has `provider_order_id`)
In progress	Provider started delivery
Completed	Fully delivered
Partial	Partly delivered. Refund `remains / quantity * charge`
Canceled	Refund the full charge
Failed	Provider rejected it or an API error. Refund and log the error

Jobs:

- `SendOrderToProvider`: calls `add`, saves `provider_order_id`, retries on network errors. If the provider returns an error, mark Failed and refund.
- `SyncOrderStatuses` (scheduled every 1–5 min): batches up to 100 pending order IDs per `status` call and updates local orders.
- `SyncServices` (scheduled, e.g. hourly): calls `services` and updates provider rate, min, max, and flags. Re-applies markup. Disables services the provider removed.
- `CheckProviderBalance`: alerts the admin when the provider balance runs low.

6. Upstream Provider API (JustAnotherPanel)

This is the standard "Perfect Panel v2" API. Most providers are compatible, so one client class can serve all of them.

- **URL:** `https://justanotherpanel.com/api/v2`
- **Method:** `POST`
- **Response:** JSON
- **Auth:** `key` parameter in every request. Keep it in `.env` or the encrypted `providers` table. Never commit it or expose it to the browser.

6.1 Service list

Request: `key`, `action=services`

```
[
  {"service": 1, "name": "Followers", "type": "Default", "category": "First Category",
   "rate": "0.90", "min": "50", "max": "10000", "refill": true, "cancel": true},
  {"service": 2, "name": "Comments", "type": "Custom Comments", "category": "Second Category",
   "rate": "8", "min": "10", "max": "1500", "refill": false, "cancel": true}
]
```

`rate` is the price per 1,000 in the account currency (USD). `type` can be `Default`, `Custom Comments`, and others, which change which fields the order needs.

6.2 Add order

Request: `key`, `action=add`, `service`, `link`, `quantity`, optional `runs`, `interval` (minutes, for drip-feed)

```
{"order": 23501}
```

6.3 Order status

Request: `key`, `action=status`, `order`

```
{"charge": "0.27819", "start_count": "3572", "status": "Partial", "remains": "157", "currency": "USD"}
```

6.4 Multiple order status

Request: `key`, `action=status`, `orders` = comma-separated IDs, up to 100.

```
{
  "1":   {"charge": "0.27819", "start_count": "3572", "status": "Partial", "remains": "157", "currency": "
  "10":  {"error": "Incorrect order ID"},
  "100": {"charge": "1.44219", "start_count": "234", "status": "In progress", "remains": "10", "currency":
}
```

6.5 Create refill

Single: `key`, `action=refill`, `order`

```
{"refill": "1"}
```

Multiple: `orders` = comma-separated IDs, up to 100.

```
[{"order": 1, "refill": 1}, {"order": 2, "refill": 2}, {"order": 3, "refill": {"error": "Incorrect order I
```

6.6 Refill status

Single: `key`, `action=refill_status`, `refill`

```
{"status": "Completed"}
```

Multiple: `refills` = comma-separated IDs, up to 100.

```
[{"refill": 1, "status": "Completed"}, {"refill": 2, "status": "Rejected"}, {"refill": 3, "status": {"erro
```

6.7 Cancel

Request: `key`, `action=cancel`, `orders` = comma-separated IDs, up to 100.

```
[{"order": 9, "cancel": {"error": "Incorrect order ID"}}, {"order": 2, "cancel": 1}]
```

6.8 Balance

Request: `key`, `action=balance`

```
{"balance": "100.84292", "currency": "USD"}
```

6.9 Integration notes

- Any response can be `{"error": "..."}`. Always check for it before reading other fields.
- Amounts are strings. Parse them as decimals and do not use floats for money.
- Provider currency is USD. Convert to INR for display and top-ups, using a configurable rate in settings.
- Use timeouts (about 15–30 s), retries with backoff, and log every request and response (with the key masked).
- Dispatch orders from a queue, not inside the HTTP request, so a slow provider never blocks the customer.
- Support multiple providers. Keep a `provider_id` per service so you can switch a service to a backup provider.

---

7. Our Own Public API (`/api/v2`)

Mirror the provider format so resellers can connect with standard tools. Authenticate with the user's `api_key`.

action	Params	Returns
`services`		service list with our (marked-up) rates
`add`	`service`, `link`, `quantity`, `runs`, `interval`	`{"order": id}`
`status`	`order` or `orders`	status object(s)
`refill` / `refill_status`	`order(s)` / `refill(s)`	refill id(s) / status
`cancel`	`orders`	per-order result
`balance`		`{"balance": "...", "currency": "INR"}`

Add per-key rate limiting and log all calls.

---

8. Admin Configuration Checklist

1. Add provider (API URL + key), then press "Test balance".
2. Import services and set the markup (e.g. +30%, or a fixed amount per 1,000).
3. Hide or rename services and group them into customer-friendly categories.
4. Configure payment gateways and webhooks.
5. Set the USD to INR rate and currency display.
6. Set up cron: `* php artisan schedule:run`, and a queue worker under Supervisor.
7. Set the low-balance alert threshold for the provider.

---

9. Security Checklist

- Provider keys encrypted at rest. Never returned to the browser.
- Wallet changes only inside DB transactions with row locks to prevent double-spend.
- Payment webhooks verified by signature and credited once (idempotent).
- Rate limit login, order, and API endpoints. Validate links and quantity against min and max.
- Admin 2FA, strong passwords, HTTPS only, regular DB backups.
- Do not log full API keys.

---

10. Build Plan and Timeline (custom Laravel build)

Phase	Work	Time
1	Setup, auth, users, wallet and ledger	1 week
2	Providers, service sync, markup, catalogue	1 week
3	Order engine, status sync, refill, cancel, refunds	1–2 weeks
4	Payments, add funds, invoices	1 week
5	Admin dashboard, tickets, own API	1–2 weeks
6	Testing, security, deployment, handover	1 week

---

11. Open Questions for the Client

- Providers to use at launch, and a backup provider?
- Which payment gateways, and does the client already hold approved merchant accounts?
- Display currency (INR only, or multi-currency)?
- Is reseller or child-panel API access needed at launch?
- Branding, domain, and who pays for hosting?